Arquitectura Front End

Actividad Sumativa 1

Diseño Arquitectonico Base del Front-End con Enfoque UX/UI

Grupo: FE7

Kevin Alexander Rodriguez Rodriguez

Docente

Jhon Rondon Suarez

Institución Universitaria Politécnico Grancolombiano

Escuela de Tecnologías de la Información y las Comunicaciones

Maestría en Arquitectura de Software

2026

Tabla de Contenido

1. Definicion del contexto y problema empresarial 3
1.1 Contexto Empresarial: “Live Bid Arena” 3
1.2 Identificacion del Problema 3
1.3 Publico Objetivo 4
1.4 Propuesta de Valor Arquitectonica 4
2. Justificacion del Framework Seleccionado: Angular 4
2.1 Gestion de Estado y Programacion Reactiva con RxJS 4
2.2 Arquitectura Modular y Mantenibilidad 5
2.3 Estrategia de Rendimiento: Deteccion de Cambios “OnPush” 5
2.4 Estándares y Gobernanza Experiencia 6
3. Diseño UX/UI y Aplicación de Leyes Cognitivas 6
3.1 Ley de Jakob: El modelo de la "Mesa de Juego" 6
3.2 Ley de Hick: Reducción de la Fatiga de Decisión 7
3.3 Ley de Fitts: Optimización de la Interacción Crítica 7
4. Gestión de Estado y Estructura de la Aplicación 7
4.1 Gestión de Estado: El patrón "Single Source of Truth" 8
4.2 Estructura Jerárquica de Componentes 8
4.3 Estrategia de Comunicación: Bidirectional Stream 9
5. Diagramas 9
6.Referencias 11

1. Definicion del contexto y problema empresarial

1.1 Contexto Empresarial: “Live Bid Arena”

El proyecto consiste en el desarrollo de "Live Bid Arena", una plataforma de subastas de próxima generación especializada en artículos de colección de alta gama (memorabilia deportiva y arte digital). A diferencia de los sitios de subastas tradicionales que funcionan mediante listados estáticos, esta aplicación introduce el concepto de "Salas de Competición en Vivo".

En este modelo, las subastas no son procesos pasivos de varios días, sino eventos síncronos de corta duración (15 a 30 minutos) donde un grupo selecto de usuarios se une a una "sala virtual" con una interfaz inspirada en las mesas de póker profesionales, permitiendo una interacción competitiva de alta intensidad.

1.2 Identificacion del Problema

A pesar del crecimiento del comercio electrónico, las plataformas de subastas actuales enfrentan tres barreras críticas que afectan la rentabilidad y la fidelización:

- **Latencia y Falta de Sincronía:** El modelo tradicional de "refrescar página" genera disparidades de tiempo. En una subasta de alto valor, una diferencia de 500 milisegundos entre lo que ve el usuario A y el usuario B destruye la integridad del proceso y genera desconfianza.
- **Carga Cognitiva Elevada (Fatiga de Decisión):** Las interfaces saturadas de datos históricos y descripciones técnicas en el momento del cierre de la subasta confunden al usuario, provocando que abandone la puja por estrés o indecisión (incumpliendo leyes básicas de usabilidad).

- **Desconexión Emocional:** Las subastas actuales carecen del componente social y la "presencia" de otros competidores, lo que reduce la adrenalina del evento y, por ende, el valor final de venta de los artículos.

1.3 Publico Objetivo

El sistema está diseñado para un perfil de usuario "Inversor Digital Activo" (25-45 años), habituado a interfaces de trading o juegos en línea, que valora la inmediatez, la transparencia de datos y la exclusividad.

1.4 Propuesta de Valor Arquitectonica

La solución propone una arquitectura Front-End basada en **Angular** que transforme la subasta en un sistema reactivo. El uso de flujos de datos constantes (Streams) permitirá que cada puja se refleje instantáneamente en todos los clientes, eliminando la sensación de latencia y convirtiendo la interfaz en un entorno de "juego" donde la toma de decisiones es fluida y natural.

2. Justificacion del Framework Seleccionado: Angular

Para el desarrollo de la "Live Bid Arena", se ha seleccionado **Angular** como el framework de arquitectura base. Esta decisión se fundamenta en la necesidad de un sistema robusto que gestione flujos de datos complejos en tiempo real, garantizando la escalabilidad y la sostenibilidad del producto a largo plazo.

2.1 Gestion de Estado y Programacion Reactiva con RxJS

La naturaleza de una subasta tipo "póker" exige una sincronización perfecta entre múltiples clientes. Angular integra de forma nativa **RxJS**, lo que permite:

Este enfoque es especialmente adecuado para el caso debido a que:

- **Manejo de Streams:** Tratar las actualizaciones de precios y cronómetros provenientes de WebSockets como flujos de datos continuos (Observables), permitiendo que la interfaz reaccione instantáneamente a cada puja sin necesidad de recargas manuales.
- **Operadores de Transformación:** Implementar lógica compleja de filtrado y combinación de datos (como validar una puja mientras se actualiza el temporizador) de manera declarativa y eficiente.

2.2 Arquitectura Modular y Mantenibilidad

Siguiendo los principios de la Unidad 1 sobre la organización del código, Angular ofrece una estructura que facilita el crecimiento del sistema:

- **Inyección de Dependencias (DI):** Permite desacoplar la lógica de comunicación con el servidor (servicios) de la representación visual (componentes). Esto facilita la realización de pruebas unitarias y el mantenimiento futuro del código.
- **Tipado Estricto con TypeScript:** Al manejar transacciones y apuestas en tiempo real, el uso de TypeScript reduce drásticamente los errores en tiempo de ejecución, asegurando que la lógica de negocio sea predecible y segura.

2.3 Estrategia de Rendimiento: Deteccion de Cambios “OnPush”

Dada la alta frecuencia de actualizaciones en una sala de subastas activa, se utilizará la estrategia de detección de cambios OnPush.

- **Optimización de Recursos:** Esta estrategia instruye al framework para que solo verifique y renderice los componentes cuyos datos de entrada hayan cambiado realmente.

- **Fluidez de la Interfaz:** Esto garantiza que, incluso con 50 participantes pujando simultáneamente, la interfaz se mantenga ágil y no consuma recursos innecesarios del navegador del usuario.

2.4 Estándares y Gobernanza Experiencia

La elección de Angular permite el uso de **Angular Material**, lo que asegura el cumplimiento de la **Ley de Jakob** al utilizar componentes que siguen estándares de diseño reconocidos globalmente (Material Design). Esto garantiza que la arquitectura visual sea consistente y profesional desde su base estructural.

3. Diseño UX/UI y Aplicación de Leyes Cognitivas

El diseño de la interfaz de "Live Bid Arena" trasciende lo estético para convertirse en una herramienta funcional que gestiona la carga mental del usuario. Basándonos en la lectura fundamental de la Unidad 2, se han aplicado tres leyes fundamentales de la psicología cognitiva para optimizar la toma de decisiones en entornos de alta presión.

3.1 Ley de Jakob: El modelo de la "Mesa de Juego"

La **Ley de Jakob** establece que los usuarios prefieren sitios que funcionan igual a los que ya conocen.

- **Aplicación**: Se ha diseñado la sala de subastas siguiendo el patrón visual de una mesa de **póker virtual.** Al colocar el objeto a subastar en el centro y a los competidores (avatares) alrededor, el usuario identifica instantáneamente quién es el líder de la puja y cuál es el estado de la partida.
- **Beneficio Arquitectónico:** Reduce la curva de aprendizaje al 0%, permitiendo que el usuario se enfoque en la estrategia de la subasta desde el primer segundo.Ecosistema maduro.

3.2 Ley de Hick: Reducción de la Fatiga de Decisión

Según la **Ley de Hick**, el tiempo que tarda una persona en tomar una decisión aumenta logarítmicamente con el número de opciones.

- **Aplicación**: En los momentos críticos (cuando el cronómetro marca menos de 2 minutos), la interfaz de Angular utiliza renderizado condicional para ocultar detalles secundarios (descripciones largas, historial completo). En su lugar, resalta únicamente tres botones de incremento rápido de puja (**+$10, +$50, +$100**).
- **Beneficio:** Al limitar las opciones, evitamos que el usuario se bloquee por "parálisis por análisis", fomentando una participación más fluida y competitiva.

3.3 Ley de Fitts: Optimización de la Interacción Crítica

la **Ley de Fitts**, indica que el tiempo para alcanzar un objetivo depende de la distancia y el tamaño de este.

- **Aplicación**: no será un elemento pequeño perdido en una esquina. Se diseñará como un área de impacto generosa ubicada en el centro de la zona de control. Además, implementaremos **"Áreas de Escucha Activa"**: aunque el usuario no haga clic exactamente en el texto del botón, toda la región circundante reaccionará al clic.
- **Beneficio:** Minimiza los errores de precisión motriz, asegurando que el usuario no pierda una subasta por un fallo al intentar pulsar el botón en el último segundo.

4. Gestión de Estado y Estructura de la Aplicación

La arquitectura de "Live Bid Arena" se basa en un flujo de datos unidireccional y una estructura de componentes desacoplada, garantizando que la interfaz sea un reflejo fiel y en tiempo real del servidor de subastas.

4.1 Gestión de Estado: El patrón "Single Source of Truth"

En una subasta de alta velocidad, la consistencia es innegociable. Si el precio sube, debe actualizarse simultáneamente en el marcador central, en el botón de puja y en el historial.

- **Implementación:** Utilizaremos **Servicios de Angular con BehaviorSubjects** (o NgRx para escalabilidad). Esto permite que el estado de la subasta sea "observable" por cualquier componente.
- **Sincronización:** Cuando llega un mensaje vía WebSocket informando una nueva puja, el servicio actualiza el estado global. Gracias a la reactividad de Angular, todos los componentes suscritos se actualizan automáticamente sin lógica adicional, eliminando errores de desfase visual.

4.2 Estructura Jerárquica de Componentes

Para asegurar la mantenibilidad (Unidad 1), la interfaz se divide en componentes especializados que siguen el principio de responsabilidad única:

1. **ArenaContainer (Smart Component):** Es el cerebro de la sala. Se encarga de conectarse al socket, recibir los datos y distribuirlos a los hijos. No tiene estilos complejos, solo gestiona lógica.
2. **AuctionTable (Dumb Component):** Representa visualmente la mesa. Recibe la lista de participantes y el objeto central. Su única función es renderizar la "escena" de la subasta.
3. **BidConsole (Dumb Component):** El panel inferior exclusivo para el usuario. Aquí es donde aplicamos la Ley de Hick y la Ley de Fitts, mostrando los botones de acción rápida.

4. **LiveHistory (Dumb Component):** Una barra lateral que muestra el "feed" de eventos. Utiliza el pipe | async de Angular para procesar los datos directamente desde el flujo del servidor, optimizando la memoria.

.

4.3 Estrategia de Comunicación: Bidirectional Stream

Al ser una aplicación **Web-Only**, aprovechamos la capacidad del navegador para mantener túneles de datos abiertos de forma persistente:

- **WebSocket Gateway:** La arquitectura no solicita datos al servidor (polling), sino que el servidor "empuja" (push) la información a Angular.
- **Interceptor de Latencia:** Implementaremos un interceptor que monitorea la calidad de la conexión del usuario. Si la latencia sube, la UI puede avisar al usuario con un cambio de color en el borde de la mesa, manteniendo la Gobernanza Experiencial mencionada en la Unidad 2.

5. Diagramas

Este diagrama de secuencia describe la columna vertebral técnica de la aplicación. La descripción debe enfatizar que la comunicación no es lineal, sino circular y reactiva. Cuando el usuario interactúa con la UI, no hay una espera pasiva; se inicia un ciclo de mensajería asíncrona mediante WebSockets. Lo más relevante para el análisis es cómo el BidService de Angular actúa como mediador, enviando la puja al servidor y, casi en simultáneo, actualizando el estado global mediante un BehaviorSubject. Esto garantiza que la interfaz sea "viva": el precio que ve el usuario es el resultado de una sincronización constante con el backend.

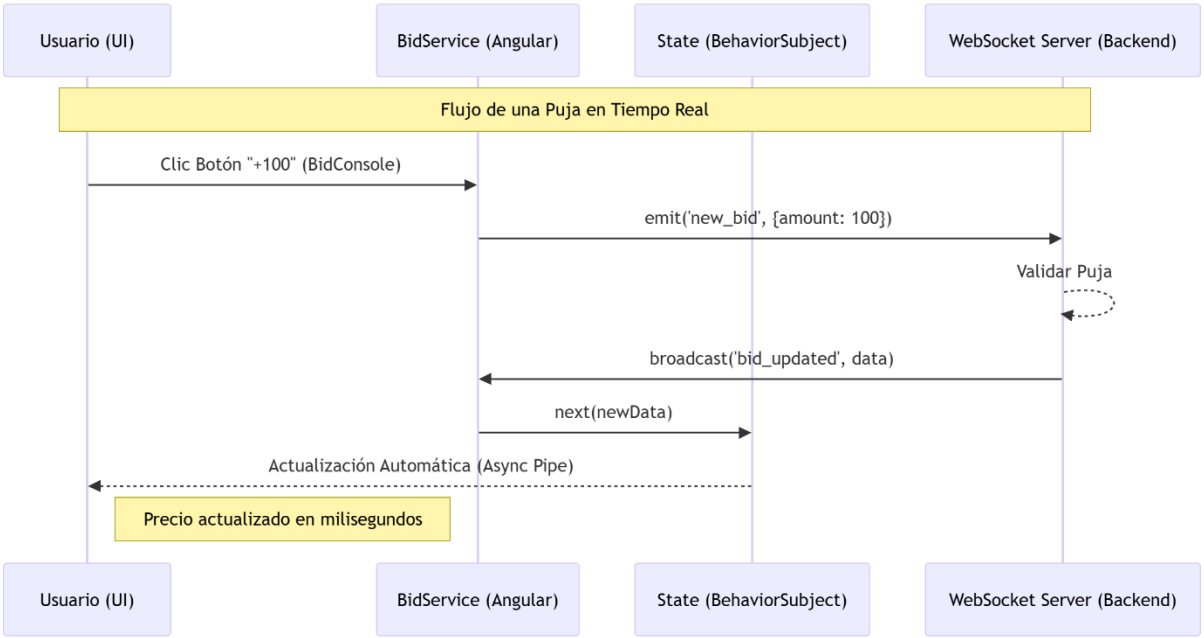

Imagen 1Flujo de la puja, Diagrama realizado con Mermaid

Esta visualización representa la **fundamentación psicológica** de la interfaz. En lugar de elegir colores por estética, el sistema utiliza una jerarquía que nace de las leyes de la percepción humana. La descripción en tu documento debe explicar que la capa superior (Psicología) actúa como el "sensor" del negocio; por ejemplo, la **Ley de Hick** dicta que, al haber poco tiempo, la interfaz debe ofrecer opciones mínimas y claras. Estos principios se filtran hacia los **Design Tokens**, que son las variables técnicas (como el color #28A745 para el éxito) que Angular utiliza para construir los componentes.

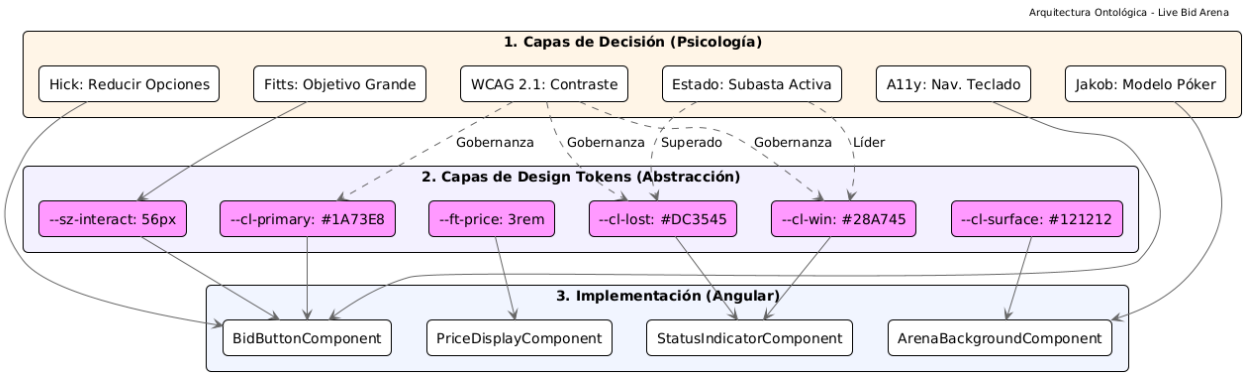

Imagen 2 Ontologia de Diseño Diagrama realizado con PlantUML

La arquitectura técnica de la aplicación se fundamenta en un esquema de **módulos altamente cohesivos y débilmente acoplados**. El diagrama de componentes ilustra cómo se separa la lógica de transporte de datos (Core Module) de la lógica de representación visual (Feature Module).

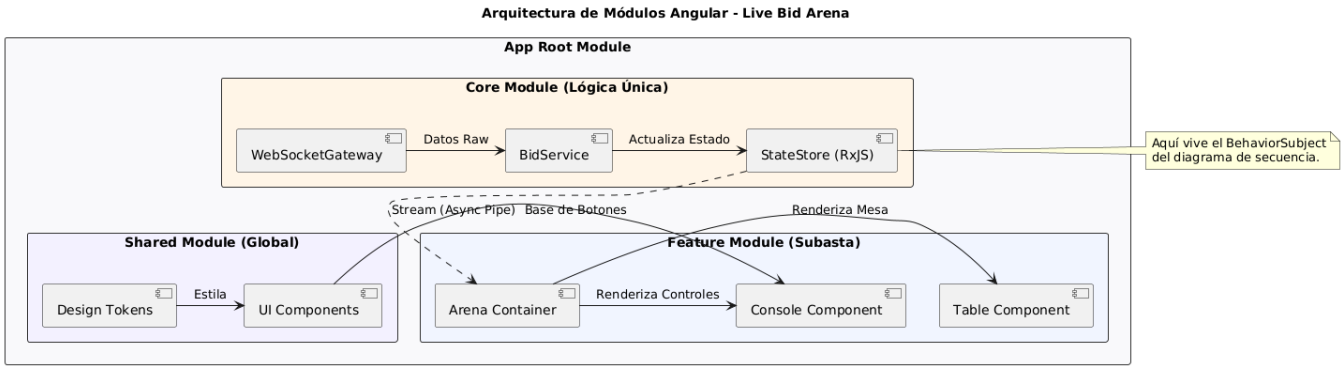

Imagen 2 Arquitectura de Software Diagrama realizado con PlantUML

6.Referencias

Design tokens – Material Design 3. (s. f.). Material Design. https://m3.material.io/foundations/design-tokens/overview

Yablonski, J. (s. f.). *Home | Laws of UX*. Laws Of UX. https://lawsofux.com/

RXJS. (s. f.). https://rxjs.dev/guide/observable

Poker Online | Play Poker Games at PokerStars™.net. (s. f.). Poker Online | Play Poker Games At PokerStars™.Net. https://www.pokerstars.com/

Google Search. (s. f.). https://www.google.com/search?q=https://v18.angular.dev/guide/architecture